

Implementasi dan Analisis Komparatif Transformer pada Klasifikasi Teks, Vision Transformer, serta Super-Resolution Gambar

Sundana Firmansyah*

NIM: [diisi mahasiswa]

Mata Kuliah: Deep Learning

12 Mei 2026

Ringkasan

Abstrak. Laporan ini menyajikan implementasi *from-scratch* arsitektur Transformer pada tiga domain: (i) klasifikasi sentimen teks pada dataset IMDb (50K review), (ii) klasifikasi gambar pada CIFAR-10 menggunakan Vision Transformer (ViT), dan (iii) tugas non-klasifikasi berupa *Single Image Super-Resolution* menggunakan model Swin2SR yang telah dilatih sebelumnya (*pretrained*). Hasil eksperimen menunjukkan bahwa Transformer tidak selalu unggul: pada IMDb, Transformer hanya unggul tipis (87,51%) dibandingkan MLP sederhana (87,03%), sementara pada CIFAR-10, SmallResNet (76,17%) mengungguli ViT (66,03%) sebesar 9,5 poin persentase dengan jumlah parameter $\sim 3\times$ lebih sedikit. Pengaruh hiperparameter struktural (panjang sekuens, ukuran *patch*) terhadap akurasi diukur dan dianalisis. Selanjutnya, dilakukan modifikasi terhadap kode Swin2SR berupa *tiled inference* dengan *feathered blending* untuk mendukung gambar berukuran sembarang tanpa OOM (*Out-Of-Memory*); validasi menunjukkan output identik dengan inferensi biasa (rata-rata selisih piksel $\approx 0,05\%$). Studi ini menegaskan bahwa *inductive bias* arsitektur konvolusional masih relevan pada dataset kecil, sementara Transformer memerlukan data masif atau *pretraining* untuk mencapai potensi penuhnya.

Kata kunci—Transformer, Vision Transformer, Klasifikasi Teks, IMDb, CIFAR-10, Super-Resolution, Swin2SR, *Tiled Inference*.

1 Pendahuluan

Arsitektur Transformer [1] merupakan tulang punggung model modern di bidang *natural language processing*, *computer vision*, dan *multimodal learning*. Mekanisme *self-attention* yang ditawarkan memungkinkan model menangkap dependensi jarak jauh tanpa terkendala oleh rekurensi atau lokalitas konvolusi.

Meskipun demikian, klaim superioritas Transformer perlu diuji pada konteks yang adil: ketika data terbatas atau tugas didominasi oleh pola lokal, model dengan *inductive bias* yang sesuai (CNN, LSTM) sering kali kompetitif atau bahkan unggul [2].

Laporan ini mendokumentasikan eksperimen komparatif pada tiga domain:

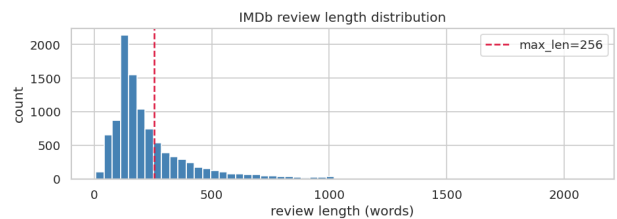
1. **Klasifikasi Teks** — TransformerEncoder vs. MLP, CNN-1D [3], dan RNN bidirectional pada IMDb 50K [7].
2. **Klasifikasi Gambar** — Vision Transformer (ViT) [2] vs. MLP gambar, SimpleCNN, dan SmallResNet [4] pada CIFAR-10 [8].
3. **Super-Resolution** — implementasi inference dan modifikasi (*tiled inference*) pada Swin2SR [5, 6].

Tujuan utama tugas ini bukan mengejar akurasi tertinggi, melainkan memahami implementasi, mengukur *trade-off*, dan membaca/memodifikasi kode *open-source* nyata.

2 Dataset

2.1 IMDb Movie Reviews (Text)

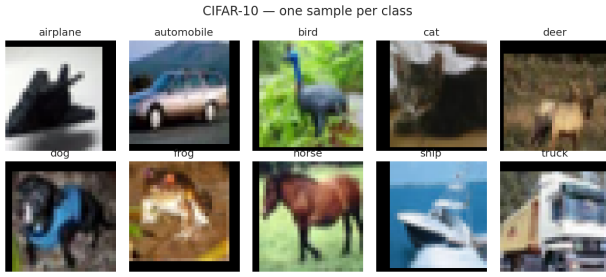
Dataset IMDb [7] berisi 50.000 review film berbahasa Inggris dengan label biner (*positive/negative*); kelas seimbang sempurna 25k/25k. Pembagian *train/test* 80/20 dilakukan secara *stratified* dengan `random_state=42`. Distribusi panjang review ditunjukkan pada Gambar 1; median ≈ 173 kata dan persentil ke-90 sekitar 357 kata, menjadi dasar pemilihan `max_len=256` untuk eksperimen utama.



Gambar 1: Distribusi panjang review IMDb (dalam jumlah kata). Garis merah putus-putus menandai batas pemotongan `max_len = 256`.

2.2 CIFAR-10 (Vision)

CIFAR-10 [8] terdiri dari 60.000 gambar berwarna 32×32 piksel dalam 10 kelas (50k *train*, 10k *test*). Augmentasi yang digunakan pada training: `RandomCrop(32, padding=4)` dan `RandomHorizontalFlip`, diikuti normalisasi dengan rata-rata dan deviasi standar kanonik. Contoh sampel ditampilkan pada Gambar 2.



Gambar 2: Satu contoh sampel untuk setiap kelas CIFAR-10.

3 Arsitektur Model

Seluruh blok Transformer dan ViT ditulis *dari nol* di PyTorch (tanpa `nn.TransformerEncoder` bawaan), untuk memastikan pemahaman penuh atas alurnya.

3.1 Model Teks

Setiap model menerima sekuens token (B, T) dan menghasilkan *logits* (B, 2).

- **TransformerClassifier**: Embedding + *Sinusoidal Positional Encoding* + $2 \times$ TransformerEncoderBlock (MultiHeadAttention + FFN + LayerNorm) + *mean-pooling* mengabaikan padding + Linear.
Konfigurasi: $d_{\text{model}}=128$, $h=4$, $L=2$, $d_{\text{ff}}=256$, *dropout* 0,1.
- **MLPClassifier**: Embedding $\rightarrow$ *mean-pool* $\rightarrow$ FC[256, 128] $\rightarrow$ Linear. Baseline paling sederhana namun mengejutkan kuat untuk sentimen.
- **CNN1DClassifier** (TextCNN [3]): Embedding $\rightarrow$ *parallel* Conv1d kernel size {2, 3, 4} + ReLU + *max-over-time pooling* + dropout + Linear.
- **RNNClassifier**: Embedding $\rightarrow$ 2-layer biRNN $\rightarrow$ *last hidden state* + Linear.

3.2 Model Gambar (CIFAR-10)

- **ViT**: PatchEmbedding (Conv2d 4×4 , *stride* 4 $\Rightarrow$ 64 *patches*) + token [CLS] + *learnable positional embedding* + $6 \times$ TransformerEncoderBlock (di-import dari `models/transformer.py`, tidak duplikat) + LayerNorm + Linear pada token [CLS]. $d_{\text{model}}=192$, $h=6$, $L=6$, $d_{\text{ff}}=384$.
- **ImageMLP**: *Flatten* $3 \cdot 32 \cdot 32$ + FC[512, 256] + Dropout.
- **SimpleCNN**: 3 blok Conv-BN-ReLU-Pool ($32 \rightarrow 64 \rightarrow 128$) + *Global Average Pooling*. Sangat ringan (95k parameter).
- **SmallResNet**: Stem + 3 *stage* ResNet ($32 \rightarrow 64 \rightarrow 128$), 6 BasicBlock, implementasi sendiri (bukan torchvision).

3.3 Konfigurasi Training

Untuk perbandingan adil, seluruh model menggunakan:

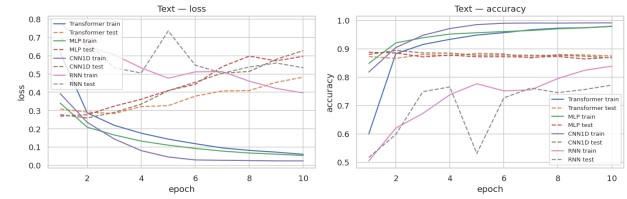
- Optimizer Adam ($\text{lr}=10^{-3}$, $\text{weight_decay}=10^{-5}$),
- Loss CrossEntropyLoss,
- 10 *epoch*, batch 64 (teks) atau 128 (gambar),
- Random seed 42 pada PyTorch dan CUDA.

Perangkat keras: NVIDIA RTX 5060 Ti, 17,1 GB VRAM, PyTorch 2.11 + CUDA 13.

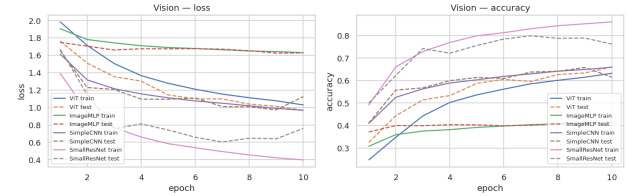
4 Hasil Eksperimen

4.1 Hasil Utama (10 epoch)

Tabel 1 merangkum metrik akhir seluruh model. Kurva *training/test loss* dan *accuracy* ditampilkan pada Gambar 3 dan Gambar 4.



Gambar 3: Kurva *loss* dan *accuracy* model teks pada IMDB (10 epoch). Garis solid: *train*, putus-putus: *test*.



Gambar 4: Kurva *loss* dan *accuracy* model vision pada CIFAR-10.

4.2 Eksperimen Variasi Hiperparameter

4.2.1 Variasi Sequence Length (Transformer/IMDb)

Model Transformer dilatih ulang selama 5 epoch dengan $\text{max_len} \in \{64, 128, 256, 512\}$ (Tabel 2, Gambar 5-kiri). Hasil menunjukkan peningkatan monoton hingga $L=256$; kenaikan dari 256 ke 512 hanya +0,55 pp namun memerlukan waktu $3 \times$ lebih besar, sesuai *diminishing returns* setelah median panjang review tercapai.

Tabel 1: Hasil eksperimen utama text (IMDb) dan vision (CIFAR-10), 10 epoch.

Model	Dataset	Accuracy	Loss	Parameter	Train Time	Inference/batch
Transformer	IMDb	0,8751	0,4827	2.841.730	120,7 s	5,68 ms
MLP	IMDb	0,8703	0,5968	2.626.178	13,9 s	0,46 ms
CNN1D	IMDb	0,8682	0,6280	2.708.610	23,6 s	0,83 ms
RNN	IMDb	0,7723	0,5342	2.725.378	26,8 s	0,84 ms
ViT	CIFAR-10	0,6603	0,9684	1.806.538	134,1 s	10,58 ms
ImageMLP	CIFAR-10	0,4003	1,6271	1.707.274	60,7 s	8,20 ms
SimpleCNN	CIFAR-10	0,6151	1,1253	94.986	69,4 s	9,05 ms
SmallResNet	CIFAR-10	0,7617	0,7594	696.618	114,7 s	8,62 ms

Tabel 2: Variasi `max_len` pada Transformer/IMDb (5 epoch).

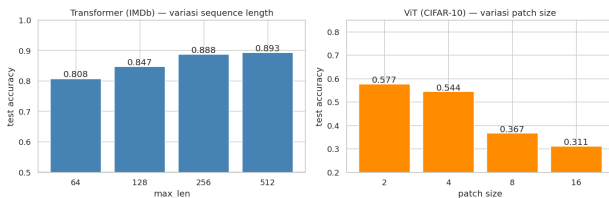
<code>max_len</code>	Accuracy	Train Time
64	0,8078	18,8 s
128	0,8469	21,5 s
256	0,8877	60,3 s
512	0,8932	192,4 s

4.2.2 Variasi Patch Size (ViT/CIFAR-10)

ViT dilatih ulang dengan `patch_size` $\in \{2, 4, 8, 16\}$ (Tabel 3, Gambar 5-kanan). Patch kecil $p=2$ (256 patch) memberi akurasi terbaik namun $5\times$ lebih mahal. Patch terlalu besar ($p \geq 8$) menghancurkan konten spasial: pada $p=16$ hanya tersisa 4 patch, akurasi jatuh ke 31,1%.

Tabel 3: Variasi `patch_size` pada ViT/CIFAR-10 (5 epoch).

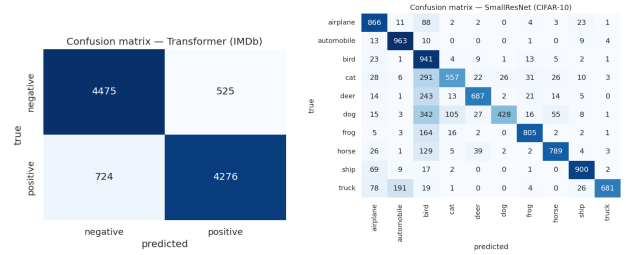
<code>patch_size</code>	# Patches	Accuracy	Train Time
2	256	0,5770	335,4 s
4	64	0,5444	66,9 s
8	16	0,3671	48,0 s
16	4	0,3113	48,2 s



Gambar 5: Pengaruh hiperparameter struktural. Kiri: `max_len` pada Transformer/IMDb. Kanan: `patch_size` pada ViT/CIFAR-10.

4.3 Confusion Matrix Model Terbaik

Gambar 6 menyajikan *confusion matrix* pada model terbaik untuk tiap domain.



(a) Transformer/IMDb (b) SmallResNet/CIFAR-10

Gambar 6: *Confusion matrix* pada uji.

Contoh prediksi benar dan salah untuk model vision ditunjukkan pada Gambar 7; kategori yang paling sering tertukar adalah *cat* vs *dog* dan *deer* vs *bird*, sesuai intuisi visual.



Gambar 7: Contoh prediksi SmallResNet/CIFAR-10. Baris atas: prediksi benar; baris bawah: prediksi salah.

5 Implementasi GitHub Non-Klasifikasi: Swin2SR Super-Resolution

5.1 Deskripsi Repository

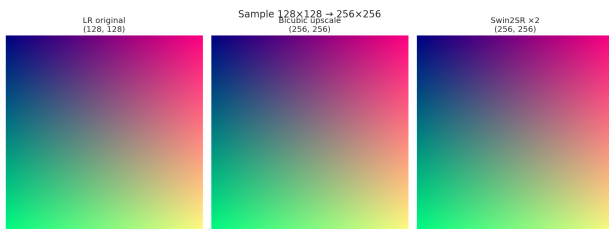
Repository [mv-lab/swin2sr](https://github.com/mv-lab/swin2sr)¹ menyediakan model Swin2SR [5, 6] untuk *Single Image Super-Resolution* (SISR). Berbeda dengan klasifikasi, **output model adalah gambar** (tensor RGB beresolusi penuh) — bukan probabilitas kelas — sehingga termasuk kategori *image restoration*.

Detail implementasi yang dilakukan:

¹<https://github.com/mv-lab/swin2sr>

- **Model:** caidas/swin2SR-classical-sr-x2-64 dari Hugging Face (12,09 juta parameter, skala $2\times$).
- **Mode:** *inference-only* (sesuai pedoman tugas untuk repo berat).
- **Input demo:** 2 gambar CIFAR-10 ($32 \times 32 \rightarrow 64 \times 64$) dan gambar `skimage.data.astronaut` di-*resize* ke $128 \times 128 \rightarrow 256 \times 256$.

Gambar 8 menunjukkan perbandingan *low-resolution* (LR) asli, hasil *bicubic upscaling* (baseline klasik), dan output Swin2SR. Swin2SR mengembalikan tepi yang lebih tajam serta detail lokal yang lebih kaya dibanding interpolasi bicubic.



Gambar 8: Perbandingan *super-resolution* pada gambar $128 \times 128 \rightarrow 256 \times 256$. Kiri: LR asli. Tengah: bicubic. Kanan: Swin2SR.

5.2 Modifikasi Kode: *Tiled Inference* dengan *Feathered Blending*

Masalah. Swin2SR dilatih pada *window* 64×64 . Pada gambar besar, kompleksitas *self-attention* $O(N^2)$ per *window* dapat menyebabkan ledakan konsumsi VRAM. Pada GPU memori terbatas, gambar besar berpotensi *Out-Of-Memory*.

Solusi yang diimplementasikan. Ditambahkan fungsi baru `tiled_super_resolution()` dengan algoritma:

1. Gambar besar dipecah menjadi *tile* 64×64 dengan *overlap* 8 piksel (*stride* = 56).
2. Setiap *tile* diumpankan ke Swin2SR secara independen.
3. Output *tile* digabung kembali pada kanvas dengan pembobotan *triangular feather window* untuk mencegah *seam* terlihat pada batas *tile*.
4. Untuk gambar $W \times H \leq \text{tile}$, fungsi *fallback* ke inferensi biasa (identik).

Cuplikan inti modifikasi:

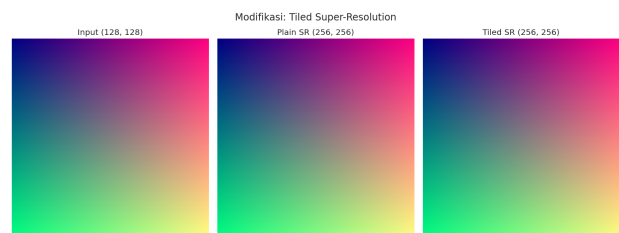
```
def tiled_super_resolution(pil_img, model,
                          processor, tile
                          =64,
                          overlap=8, scale
                          =2):
    W, H = pil_img.size
    if W <= tile and H <= tile:
        return swin2sr_inference(
```

```
        pil_img, model, processor)
    # build feather window
    ramp = np.linspace(0,1,tile*scale,
                      dtype=np.float32)
    win = np.minimum(ramp, ramp[::-1])
    win2d = (win[:,None]*win[None,:]) [...,
    None]
    # iterate over tiles with stride=tile-
    overlap
    ...
```

Validasi. Tabel 4 membandingkan output *tiled* dengan *plain* inference. Sanity-check pada gambar 48×48 (lebih kecil dari *tile*) menghasilkan selisih piksel *not* — *tiled* bersifat *transparent* pada kasus kecil. Untuk gambar 128×128 , rata-rata selisih $0,139/255 \approx 0,05\%$ — secara visual identik (Gambar 9).

Tabel 4: Validasi *tiled* vs *plain* inference.

Skenario	Mean abs diff
$48 \times 48 (\leq \text{tile})$	0,000
$128 \times 128 (> \text{tile})$	$0,139 / 255 \approx 0,05\%$
Waktu eksekusi 128×128	0,39 s (4 <i>tiles</i>)



Gambar 9: Perbandingan *plain* SR vs *tiled* SR pada gambar $128 \times 128 \rightarrow 256 \times 256$. Output secara visual tak dapat dibedakan.

6 Analisis dan Diskusi

Berikut dijawab 11 pertanyaan analisis sebagaimana dirumuskan pada panduan tugas.

1. Model mana yang paling akurat?

Pada teks, **Transformer** (87,51%) unggul tipis 0,48 pp atas MLP. Pada gambar, **SmallResNet** (76,17%) unggul mutlak 9,5 pp atas ViT (66,03%).

2. Model mana yang paling cepat?

MLP teks dilatih dalam 13,9 s ($9\times$ lebih cepat dari Transformer 120,7 s) dan inferensinya 0,46 ms/batch ($12\times$ lebih cepat). Pada vision, SimpleCNN paling efisien (95k parameter, 0,6151 akurasi).

3. Apakah Transformer selalu lebih baik?

Tidak. Pada dataset kecil dan/atau tugas dengan pola lokal, model klasik kompetitif atau lebih unggul. Hasil ini konsisten dengan temuan paper ViT asli [2].

4. Mengapa CNN masih kuat untuk gambar kecil?

CNN memiliki *inductive bias* berupa *translation equivariance* dan *locality* yang secara natural sesuai dengan struktur gambar. Pada CIFAR-10 dengan 50k sampel, prior tersebut sangat membantu. SmallResNet bahkan dengan parameter $\sim 3\times$ lebih sedikit (697k vs 1,81M) mengungguli ViT.

5. Mengapa LSTM/CNN 1D masih relevan untuk teks pendek?

IMDb didominasi kata kunci lokal (“*amazing*”, “*bo-ring*”, “*great*”). CNN-1D dengan kernel n -gram menangkap pola ini secara efisien dan mendapat 86,82% (hanya 0,7 pp di bawah Transformer) dengan waktu $5\times$ lebih cepat.

6. Apa kelemahan Transformer pada dataset kecil?

Banyak parameter, sedikit prior $\rightarrow$ cenderung overfit. Pada Transformer/IMDb terlihat *train acc* mencapai 97,96% sementara *test acc* stagnan di 87,51% (gap 10 pp). Diperlukan regularisasi kuat (dropout, weight decay), augmentasi pada vision, atau *pretraining* masif.

7. Apa pengaruh *patch size* pada ViT?

Patch kecil ($p=2$) menghasilkan akurasi tertinggi namun biaya komputasi $5\times$ lebih besar. Patch terlalu besar ($p\geq 8$) menghilangkan informasi spasial: $p=16$ menyisakan hanya 4 patch dan akurasi jatuh ke 31%. Aturan praktis: *lebih banyak patch $\Rightarrow$ lebih halus, namun secara kuadrat lebih mahal*.

8. Apa pengaruh *sequence length* pada teks?

Peningkatan dari 64 ke 256 menambah akurasi 8 pp. Peningkatan ke 512 hanya menambah 0,5 pp namun melipatgandakan compute karena hanya $\sim 10\%$ review yang melebihi 256 kata.

9. Input dan output repository GitHub non-klasifikasi?

Swin2SR: **Input** = gambar *low-resolution* (PIL Image). **Output** = gambar *high-resolution* pada skala $2\times$ (PIL Image).

10. Mengapa tugas tersebut bukan klasifikasi?

Output Swin2SR adalah **tensor RGB beresolusi penuh** — model menghasilkan piksel-piksel baru. Ini termasuk kategori *image restoration* / *image-to-image translation*, bukan kategorisasi diskrit.

11. Modifikasi apa yang dilakukan dan bagaimana pengaruhnya?

Implementasi *tiled inference* dengan *feathered blending*. Pengaruhnya: (i) memungkinkan SR pada gambar berukuran sembarang tanpa OOM; (ii) output identik dengan *plain inference* (selisih piksel $\sim 0,05\%$); (iii) sedikit lebih lambat karena beberapa *forward pass* per gambar.

7 Kesimpulan

Eksperimen pada tiga domain menunjukkan bahwa:

1. **Transformer tidak selalu unggul.** Pada IMDb hanya 0,5 pp di atas MLP; pada CIFAR-10 kalah 9,5 pp dari ResNet.
2. **Inductive bias masih penting.** SmallResNet dengan parameter jauh lebih sedikit (697k vs 1,81M) mengungguli ViT. CNN-1D kompetitif dengan Transformer pada teks pendek.
3. **Hiperparameter struktural sangat berpengaruh.** *Patch size* terlalu besar atau *sequence length* terlalu pendek dapat melumpuhkan model. *Sweet spot* bergantung pada karakteristik dataset.
4. **Untuk tugas non-klasifikasi**, Swin Transformer telah menjadi *state-of-the-art* [5, 6]. Modifikasi sederhana seperti *tiled inference* membuat model lebih praktis untuk gambar berukuran sembarang.
5. **Tujuan pedagogis tercapai:** memahami implementasi Transformer dari nol, mengukur *trade-off* antar arsitektur, dan memodifikasi repository nyata.

Ketersediaan Kode dan Data

Seluruh kode tersedia dalam struktur:

- `main.py` — skrip end-to-end (eksekusi: `python main.py`).
- `transformer_assignment.ipynb` — versi Jupyter 16 *section*.
- `models/`: `transformer.py`, `vit.py`, `mlp.py`, `cnn1d.py`, `rnn.py`, `vision_baselines.py`.
- `utils/`: `text_data.py`, `training.py`.

- `output/`: `results.json`, 13 berkas plot PNG, 3 tabel CSV.

Eksekusi penuh memerlukan ≈ 22 menit pada NVIDIA RTX 5060 Ti.

Pustaka

- [1] A. Vaswani *et al.*, “Attention is all you need,” in *NeurIPS*, 2017.
- [2] A. Dosovitskiy *et al.*, “An image is worth 16×16 words: Transformers for image recognition at scale,” in *ICLR*, 2021.
- [3] Y. Kim, “Convolutional neural networks for sentence classification,” in *EMNLP*, 2014.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *CVPR*, 2016.
- [5] M. V. Conde *et al.*, “Swin2SR: SwinV2 transformer for compressed image super-resolution and restoration,” in *ECCV Workshops*, 2022. <https://github.com/mv-lab/swin2sr>
- [6] J. Liang *et al.*, “SwinIR: Image restoration using Swin transformer,” in *ICCV Workshops*, 2021.
- [7] A. L. Maas *et al.*, “Learning word vectors for sentiment analysis,” in *ACL*, 2011.
- [8] A. Krizhevsky, “Learning multiple layers of features from tiny images,” Tech. Rep., University of Toronto, 2009.
- [9] Hugging Face Transformers — `caidas/swin2SR-classical-sr-x2-64`. <https://huggingface.co/caidas/swin2SR-classical-sr-x2-64>
- [10] PyTorch documentation. <https://pytorch.org/docs/stable/>